



Product Requirement Document (PRD)

1. Overview

- **Product :** Hisab Kitab
- **Author:** Anil BK
- **Date:** @May 4, 2026

2. Problem statement

- **What problem are we solving?**
Manual bookkeeping using notebooks is **slow, error-prone, and not searchable**, making it hard to track balances and get summaries quickly.
- **Why now?**
Users already use smartphones but still rely on paper. This creates a **gap between capability and tools**, making it the right time to digitize their existing workflow without changing their habits.
- **Who is impacted?**
 - **Primary:** Small shop owners / self-employed individuals managing daily financial records.
 - **Secondary:** Anyone tracking personal lending, expenses, or informal accounts.

3. Goals

- Achieve consistent daily usage, with the user recording transactions every day and fully replacing the paper notebook
- Ensure fast data entry, allowing any transaction to be recorded in under 30 seconds
- Maintain complete accuracy in balance calculations, eliminating the need for manual verification
- Provide quick access to financial information, enabling the user to check balances instantly

Non-goals

- Do not support multiple users or shared accounts in the initial version
- Do not integrate with banks or automatically import transactions
- Do not support multiple currencies; limit to a single local currency
- Do not include notifications, reminders, or automation features

4.Target users

- Primary user: Small shop owners or self-employed individuals who maintain daily financial records manually and need a simple, fast way to track transactions
- Prefer familiar mental models (e.g., "page per person" like a notebook) rather than complex interfaces
- Secondary user: Individuals managing informal personal finances, such as lending/borrowing money or tracking household expenses

5.Core Features

5.1 Account (Khata) Management

- Create and manage separate accounts for each person or category
- View all accounts with current balance at a glance

5.2 Transaction Entry

- Add transactions with amount, date, and type (given/received)

- Include optional notes for context
- Automatically update running balance after each entry

5.3 Balance Tracking

- Display real-time balance for each account
- Clearly indicate whether money is owed or to be received

5.4 Dashboard & Overview

- List all active accounts in a single view
- Highlight balances using simple visual indicators

5.5 Search

- Allow users to search across all Khatas and entries using **name or note content**

6. User stories

- As a **shop owner**, I want to create a **Khata for each customer**, so that I can track how much each person owes me.
- As a **shop owner**, I want to record a **given/received** transaction in under 30 seconds, so that I can log entries while I'm busy.
- As a **user**, I want the app to **auto-calculate the running balance** after every transaction, so that I don't have to do manual math.
- As a **user**, I want to **search by customer name or note text**, so that I can find old entries instantly.
- As a **user**, I want a simple **summary view** (total receivable vs payable), so that I understand my overall position at a glance.

7. Success metrics

Metric	Baseline	Target (MVP)	Measurement
Activation rate (create 1st Khata + add 1st transaction)	TBD	≥ 60%	Event: khata_created + transaction_added

Metric	Baseline	Target (MVP)	Measurement
Daily active usage (DAU / WAU)	TBD	≥ 40%	App opens + transaction_added
Average time to add a transaction	TBD	≤ 30 seconds	Time between entry_opened → transaction_saved
Retention (D7)	TBD	≥ 25%	Cohort retention
Search usage rate	TBD	≥ 20% of users weekly	Event: search_performed
Balance accuracy complaints	TBD	Near-zero critical issues	Support tickets / feedback tag

8. Open questions

- Should the MVP support **offline-first** usage (no internet) since many users may have inconsistent connectivity?
- What is the best **default transaction type** language: "Given/Received" vs "Debit/Credit" vs "You gave/You got"?
- Do we need **number formatting** and separators for Nepali/Indian style (e.g., 1,00,000) from day one?
- How should we handle **partial payments** and showing "remaining balance" in a way users understand?
- Should we allow **backdated entries** and what should be the default date (today vs last used)?
- What is the simplest **backup/restore** strategy (device-only vs account login) while still keeping setup easy?
- What are the minimum **security** requirements (PIN/biometric lock) for users who share phones?

9. Milestones (MVP Plan — Hisab Kitab)

The MVP should be planned as a **progressive validation system**, not just feature delivery. Each milestone should answer a key question: **Does this actually work**

for father in real usage?

9.1 Foundation Setup (System Skeleton)

What

- Backend FastAPI project setup
- PostgreSQL connection (Supabase)
- Basic project structure (models, services, repositories)
- Frontend React + Vite setup (PWA-ready scaffold)
- Authentication skeleton (JWT + phone/PIN placeholder)

Why

- Prevents architecture debt early
- Ensures full-stack pipeline works end-to-end before features

Exit criteria

- Server runs locally
- Frontend can call backend health endpoint
- DB connection established successfully

9.2 Authentication + User Access Control

What

- Phone + PIN login flow
- JWT token generation and validation
- Protected routes (backend middleware)
- Frontend login screen + token storage

Why

- Everything in the system depends on secure user context
- Needed before any financial data is introduced

Exit criteria

- User can log in and stay authenticated across refresh
- Unauthorized requests are blocked properly

9.3 Core Data Model (Khata + Entry)

What

- Khata model (PERSONAL / EXPENSE)
- Entry model (Jama / Udhar)
- Relationships (User → Khata → Entry)
- Basic CRUD APIs for Khatas and Entries

Why

- This is the **core mental model replacement of notebook pages**
- Without this, product does not exist

Exit criteria

- Can create Khata
- Can add entries inside it
- Data persists correctly in DB

9.4 Balance Engine (Core Value Feature)

What

- Backend balance calculation (Jama - Udhar)
- Running balance per entry
- Khata-wise computed balance API
- Sorted Khata listing by balance

Why

- This replaces manual calculation (biggest pain point in PRD)
- If this is wrong, entire system loses trust

Exit criteria

- Balance matches manual calculation exactly
- No frontend computation required

9.5 Basic UI (Daily Usability Layer)

What

- Home screen (Khata cards)
- Khata detail screen (entries list)
- Add entry form (fast input optimized)
- Balance display with color indicators

Why

- This is where "paper notebook feel" is replicated digitally
- UX decides adoption, not backend design

Exit criteria

- Entry flow < 30 seconds
- 2-tap access to key actions (FR requirement)
- Usable on mobile without friction

9.6 Monthly Report + Search

What

- Monthly summary API (Jama / Udhar / net flow)
- Outstanding balance report
- Global search across Khatas + entries

Why

- Converts raw tracking into **decision-making tool**
- Adds "why am I using this app instead of notebook?"

Exit criteria

- User can answer: "how was this month?" in one screen

- Search returns relevant results quickly

9.7 Polish + Reliability Layer

What

- Error handling standardization
- Loading states (especially server wake-up delay)
- Input validation hardening
- Basic performance optimization (query efficiency)

Why

- Real-world usage happens in imperfect conditions (slow internet, accidental clicks)
- Trust is built through stability, not features

Exit criteria

- No crashes in normal usage
- Clear error messages for all failures
- Smooth mobile experience on low-end devices

9.8 Deployment (Production Readiness)

What

- Deploy backend on Render
- Deploy frontend on Vercel
- Connect Supabase production DB
- Environment configuration (.env separation)

Why

- MVP is not complete until real user can use it daily
- Deployment reveals hidden integration issues

Exit criteria

- Fully working live system
- Login → Add Entry → View Balance works end-to-end